

REDES NEURONALES EN PRÁCTICA CONTROLADA

Cómo operar una red neuronal con rigor técnico

BLOQUE 0 · ENFOQUE GENERAL DE LA SESIÓN

En la sesión anterior has visto **por qué aparecen las redes neuronales**:

- el diseño manual de modelos deja de escalar,
- el *feature engineering* colapsa,
- aprender representaciones se vuelve inevitable.

En esta sesión damos el siguiente paso natural:

usar redes neuronales por primera vez de forma práctica,
pero bajo un control técnico estricto y deliberado.

El objetivo **no es rendimiento máximo**.

El objetivo es **criterio profesional**.

Aquí aprenderás a:

- entrenar una red neuronal sencilla,
- compararla con modelos clásicos,
- detectar cuándo sobreajusta,
- regularla,
- evaluar su estabilidad,
- y decidir si merece la pena usarla.

BLOQUE 1 · QUÉ ESTÁ CALCULANDO REALMENTE UNA RED NEURONAL

Antes de entrenar una red neuronal es imprescindible entender **qué se está ajustando realmente durante el entrenamiento**.

En esta sesión no vas a derivar fórmulas ni a implementar algoritmos desde cero, pero **sí vas a tomar decisiones que afectan directamente a este mecanismo interno**.

El perceptrón: la unidad básica

La unidad mínima de cualquier red neuronal es el **perceptrón**.

Un **perceptrón simple** realiza siempre el mismo cálculo:

$$z = \sum_{i=1}^n w_i x_i + b$$

y aplica una función de activación:

$$a = f(z)$$

Es decir, una neurona calcula una salida y a partir de muchas entradas x ponderadas por sus pesos.

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Donde:

- x_i son las entradas (variables del dataset),
- w_i son los pesos que se ajustan durante el entrenamiento,
- b es el sesgo (*bias*),
- f es una función de activación no lineal (por ejemplo, ReLU o sigmoide).

Un perceptrón simple **solo puede modelar relaciones lineales**.

Por sí solo, es insuficiente para la mayoría de problemas reales.

Del perceptrón simple al perceptrón multicapa (MLP)

Cuando se **conectan muchos perceptrones** organizados en capas, se obtiene un **perceptrón multicapa (MLP, Multi-Layer Perceptron)**.

Eso es exactamente el tipo de modelo que estás utilizando en esta sesión.

Una red neuronal no es más que:

- muchas de estas unidades,
 - conectadas entre sí,
 - donde la salida de una capa es la entrada de la siguiente.
-

Arquitectura de una red neuronal (lo mínimo necesario)

Una red neuronal básica se organiza en:

- **capa de entrada** → recibe las variables del problema,
- **capas ocultas** → transforman la representación interna,
- **capa de salida** → produce la predicción final.

La **arquitectura** de una red se define por:

- el número de capas ocultas,
- el número de neuronas en cada capa,
- la función de activación utilizada.

En esta sesión **no vas a diseñar arquitecturas complejas**.

Trabajarás con arquitecturas **deliberadamente pequeñas**, porque:

más capacidad implica más riesgo de sobreajuste
y menos control experimental.

Funciones de activación: por qué importan aquí

Las funciones de activación introducen **no linealidad** en el modelo.

Sin ellas, una red neuronal profunda sería equivalente a un modelo lineal,

por muchas capas que tuviera.

En esta sesión **no se comparan activaciones en profundidad**.

Se utilizan activaciones estándar porque el objetivo no es exprimir rendimiento, sino **controlar el comportamiento del modelo**.

La elección de activación afecta directamente a:

- la capacidad del modelo,
- la facilidad de entrenamiento,
- el riesgo de sobreajuste.

Todo lo que ajustes más adelante (regularización, early stopping, estabilidad) actúa **sobre este mecanismo básico**.

Notebook · Bloque 1

Entrenamiento de un **perceptrón multicapa (MLP)** con arquitectura sencilla.

Observa cómo el número de neuronas y la activación influyen en el ajuste del modelo.

A partir de este punto, toda decisión técnica que tomes en la sesión (regularización, parada temprana, estabilidad) actuará directamente sobre este mecanismo.

BLOQUE 2 · LA HERRAMIENTA: UNA RED NEURONAL COMO UN MODELO MÁS

Para esta primera práctica **no usamos frameworks de deep learning**.

Trabajamos con `MLPClassifier` de **scikit-learn**, porque:

- mantiene la misma API que ya conoces (`fit` , `predict` , `predict_proba`),
- permite comparar redes neuronales y modelos clásicos en igualdad de condiciones,
- evita que la complejidad del código oculte errores de diseño.

Regla explícita de la sesión:

Una red neuronal se evalúa como cualquier otro modelo.
No tiene privilegios por ser “neural”.

Notebook · Bloque 2

Primer entrenamiento de una red neuronal sencilla con `MLPClassifier`.

BLOQUE 3 · CONTROL DEL EXPERIMENTO (TRAIN / VALIDATION / TEST)

En redes neuronales **no puedes decidir mirando el conjunto de test**.

Por eso, el protocolo obligatorio es:

- **training set** → ajuste de pesos
- **validation set** → decisiones de arquitectura y regularización
- **test set** → evaluación final (una sola vez)

En esta sesión se utiliza:

- un *hold-out* explícito, o
- la validación interna mediante `validation_fraction`.

No se usa K-Fold aquí

porque el coste computacional de las redes lo hace inviable en exploración temprana.

Esto no es una simplificación académica.

Es una **decisión profesional realista**.

Notebook · Bloque 3

Separación explícita de los datos y análisis del impacto de decidir con validación.

BLOQUE 4 · EL BASELINE OBLIGATORIO

Antes de confiar en una red neuronal, debes compararla con un **modelo mínimo de referencia**.

Ese baseline puede ser:

- un `DummyClassifier` , o
- una `LogisticRegression` .

Regla fundamental de esta sesión:

Si la red neuronal no mejora al baseline de forma clara y estable, el coste estructural no está justificado.

Aquí no se evalúa “si funciona”.

Se evalúa **si merece la pena**.

Notebook · Bloque 4

Tabla comparativa de modelos: baseline vs red neuronal, usando las mismas métricas.

BLOQUE 5 · OVERFITTING: VERLO, NO INTUIRLO

Las redes neuronales **tienden a sobreajustar por defecto**.

Por eso, en esta sesión **no basta con mirar métricas finales**.

Se analizan **curvas de pérdida (loss curves)** para observar:

- evolución del error en entrenamiento,
- evolución del error en validación.

Criterio operativo:

Si la pérdida de entrenamiento sigue bajando y la de validación deja de mejorar o empeora, la red está aprendiendo ruido.

Aquí el sobreajuste **no se debate**.

Se **detecta visualmente**.

Notebook · Bloque 5

Visualización de curvas de pérdida y localización del punto de sobreajuste.

BLOQUE 6 · MÉTRICAS: NO SOLO ACERTAR, SINO CON CUÁNTA SEGURIDAD

Una red neuronal no solo produce clases.

Produce **probabilidades**.

En esta sesión se analiza explícitamente:

- aciertos con baja confianza (≈ 0.5),
- aciertos con alta confianza (≈ 0.9).

Interpretación clave:

Un modelo que acierta con margen es más robusto y controlable que uno que acierta por los pelos.

Notebook · Bloque 6

Uso de `predict_proba` para evaluar la confianza de las predicciones.

BLOQUE 7 · REGULARIZACIÓN: EL FRENO DE MANO

La regularización se trabaja aquí **como mecanismo real**, no como concepto abstracto.

Dos herramientas clave:

Penalización de pesos (`alpha`)

- evita soluciones frágiles,
- limita pesos excesivos,
- reduce el sobreajuste.

Early stopping

- la red se evalúa continuamente en validación,
- se detiene cuando deja de mejorar.

Interpretación profesional:

El modelo sabe cuándo debe parar
porque está empezando a memorizar.

Notebook · Bloque 7

Comparación del comportamiento del modelo con y sin regularización.

BLOQUE 8 · ESTABILIDAD: EL FACTOR OLVIDADO

Las redes neuronales son **estocásticas**:

- dependen de la inicialización,
- del orden de los datos,
- del azar interno.

Por eso, en esta sesión se entrena **la misma red varias veces** con distintas semillas.

Criterio explícito:

- variación pequeña → modelo estable
- variación grande → modelo frágil

Un modelo inestable no es profesional.
Es una lotería.

Notebook · Bloque 8

Entrenamientos repetidos con distintas semillas y análisis de la variabilidad.

BLOQUE 9 · PRÁCTICA CONTROLADA: EL RETO TÉCNICO

Reto de la sesión:

- dataset con ruido,
- red neuronal de **capacidad muy limitada**,

- objetivo: rendimiento aceptable **y estable**.

Este reto obliga a:

- mejorar datos antes que arquitectura,
- ajustar regularización con precisión,
- aceptar límites del modelo.

Notebook · Bloque 9 — Reto de la red minimalista

Optimización del modelo bajo restricciones estrictas.

BLOQUE 10 · CIERRE DE LA SESIÓN

Esta sesión no trata de demostrar que:

| “las redes neuronales funcionan”.

Eso ya lo sabías.

Trata de aprender que:

| una red neuronal solo es útil
| si puedes justificar su comportamiento
| y confiar en su estabilidad.

Ese es el estándar profesional
que se asume a partir de ahora.